



MÓDULO 6 · T17

Calidad y Deploy: EAS Build y Deploy

 *Publica tu app en tiendas con EAS*

Carlos Rodríguez Contreras · GitHub: `karrocon`

¿Qué aprenderás hoy?

En esta sesión dominarás el flujo completo de compilación y publicación de una app React Native usando EAS. Al terminar, podrás generar builds y publicar en las tiendas oficiales sin necesitar herramientas nativas instaladas localmente.

1

Flujo de build con EAS

Entender cómo EAS compila tu app en la nube sin Android Studio ni Xcode.

2

Perfiles en eas.json

Configurar perfiles de desarrollo, preview y producción adaptados a cada necesidad.

3

APK y App Bundle

Generar un APK para pruebas internas y un AAB para publicar en Google Play Store.

4

Publicar en tiendas

Usar `eas submit` para subir la app a Google Play Store o Apple App Store.

¿Qué es EAS?

EAS (Expo Application Services) es la infraestructura cloud de Expo para compilar y publicar apps React Native sin necesitar Android Studio ni Xcode instalados localmente.

☁ Build en la nube

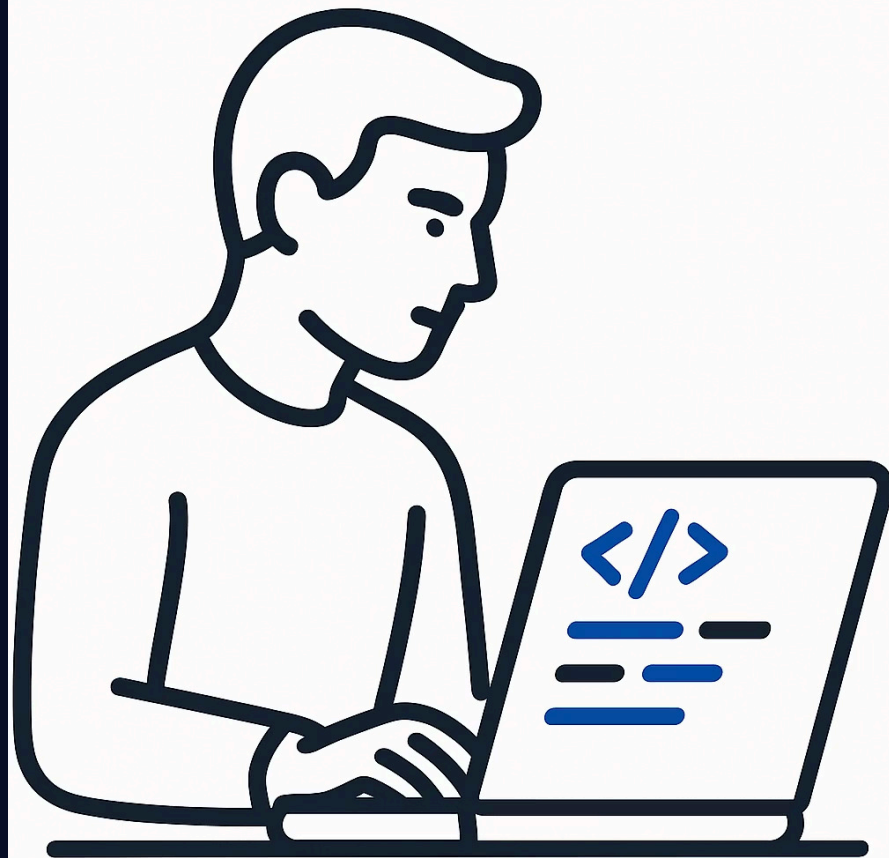
Los servidores de Expo compilan tu app por ti. No necesitas configurar entornos locales complejos.

📦 Múltiples outputs

Genera APK para pruebas, AAB para Google Play o IPA para App Store desde un único comando.

🚀 Deploy integrado

Con `eas submit` puedes subir directamente a las tiendas sin salir de la terminal.



Instalación y login

Antes de lanzar tu primera build necesitas instalar la CLI de EAS globalmente, autenticarte con tu cuenta de Expo y configurar el proyecto. El comando `eas build:configure` genera automáticamente el fichero `eas.json` con los perfiles base.

```
# Instalar EAS CLI globalmente
```

```
npm install -g eas-cli
```

```
# Login con tu cuenta Expo
```

```
eas login
```

```
# Configurar el proyecto (genera eas.json)
```

```
eas build:configure
```

 Necesitas una cuenta gratuita en **expo.dev** para poder usar EAS. El plan gratuito incluye un número limitado de builds mensuales.

eas.json — Perfiles de build

¿Por qué usar perfiles?

Cada entorno tiene necesidades distintas: el equipo de desarrollo necesita acceso a herramientas de depuración, QA necesita un APK instalable directamente, y producción necesita un bundle optimizado para la tienda.

Los perfiles en `eas.json` encapsulan esa configuración – un solo fichero, cero ambigüedad sobre qué se está compilando.

- ✓ Puedes ejecutar `eas build --profile preview` y EAS sabrá exactamente qué configuración aplicar.

Estructura de eas.json

```
{
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "android": {
        "buildType": "apk"
      },
      "distribution": "internal"
    },
    "production": {
      "android": {
        "buildType": "app-bundle"
      }
    }
  }
}
```

Los 4 perfiles de PokeApp

La PokeApp usa cuatro perfiles que cubren todos los escenarios del ciclo de vida de la app, desde el desarrollo activo hasta la distribución en Google Play Store.

Perfil	Output	Uso
development	devClient	Desarrollo activo con expo-dev-client – hot reload y herramientas de debug.
preview	APK	Pruebas internas – se puede compartir el APK directamente con el equipo de QA.
apk	APK (assembleRelease)	Distribución directa fuera de la tienda, sin pasar por Google Play.
production	AAB (app-bundle)	Publicación en Google Play Store – formato obligatorio para nuevas apps.

Comandos de build

Todos los comandos siguen el mismo patrón: `eas build --platform [android | ios | all] --profile [nombre-perfil]`. EAS se encarga del resto en sus servidores.

APK para distribuir a testers

```
eas build --platform android --profile preview
```

Bundle de producción para Google Play

```
eas build --platform android --profile production
```

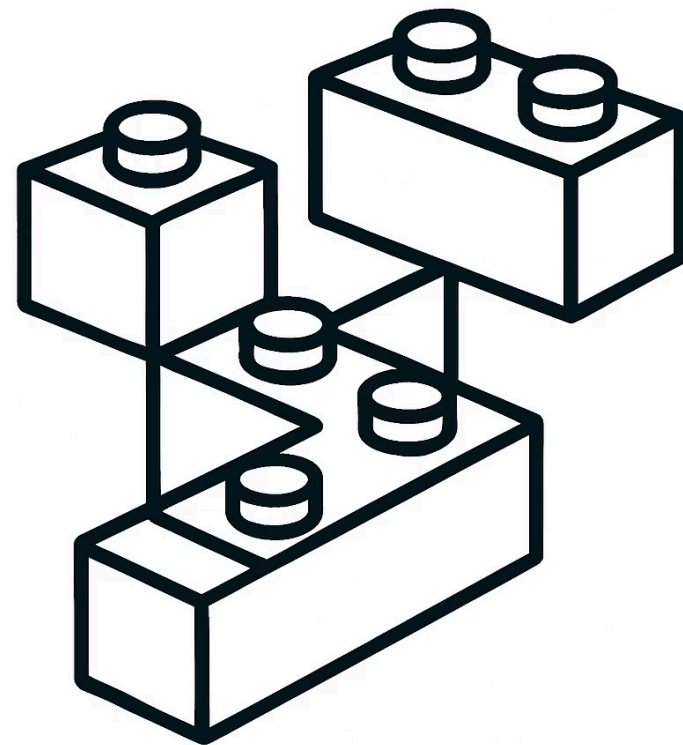
Build para iOS (requiere cuenta Apple Developer)

```
eas build --platform ios --profile production
```

Las dos plataformas a la vez

```
eas build --platform all --profile production
```

❗ El build tarda entre **5 y 15 minutos**. Puedes seguir el progreso en **expo.dev** o directamente en la terminal.



app.json — Configuración de la app

Antes de lanzar un build de producción asegúrate de que `app.json` tiene los identificadores correctos. El `package` de Android y el `bundleIdentifier` de iOS son únicos en sus respectivas tiendas – una vez publicada la app, no pueden cambiarse.

```
{
  "expo": {
    "name": "PokeApp",
    "slug": "pokeapp",
    "version": "1.0.0",
    "android": {
      "package": "com.karrocon.pokeapp",
      "versionCode": 1
    },
    "ios": {
      "bundleIdentifier": "com.karrocon.pokeapp"
    }
  }
}
```

⚠ Incrementa `versionCode` en cada subida a Google Play – si envías el mismo número, la tienda rechazará el build.

Publicar en tiendas con eas submit

Comandos de submit

```
# Subir a Google Play  
# (requiere Service Account JSON)  
eas submit --platform android
```

```
# Subir a App Store  
# (requiere Apple credentials)  
eas submit --platform ios
```

```
# Build + Submit en un solo comando  
eas build --platform android --profile production --auto-submit
```

Requisitos previos

→ Google Play

Cuenta de desarrollador (\$25 única vez) + Service Account JSON con permisos de publicación.

→ Apple App Store

Cuenta Apple Developer (\$99/año) + credenciales configuradas en EAS.

Flujo completo de deploy

El proceso de publicación sigue siempre los mismos seis pasos, desde el comando inicial hasta la app disponible para los usuarios finales.



```
graph LR; A[eas build] --> B[Compilación]; B --> C[Descargar APK/AAB]; C --> D[eas submit];
```

eas build

Compilación

Descargar APK/AAB

eas submit

El paso de revisión de la tienda es el único que no puedes controlar directamente – Google Play suele tardar entre 1 y 3 días hábiles en revisar una nueva app o actualización.

Tour starter/ — eas.json + README.md

El repositorio incluye un `eas.json` de inicio con los perfiles `development` y `preview` ya configurados. El `README.md` del starter tiene los comandos paso a paso para configurar tu primera build.

```
{
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "android": {
        "buildType": "apk"
      }
    }
  }
}
```

⚠ El starter no incluye el perfil `production` de forma intencionada – añadirlo es parte del ejercicio T17.

Ejercicio T17

Pon en práctica todo lo aprendido. Sigue los pasos en orden – cada uno construye sobre el anterior. El bonus es opcional pero muy recomendable para familiarizarte con el flujo completo de distribución.

1 Instala eas-cli y autentica

Ejecuta `npm install -g eas-cli` seguido de `eas login` con tu cuenta de Expo.

2 Configura el proyecto

En tu proyecto, ejecuta `eas build:configure` para generar el `eas.json` inicial.

3 Compara los eas.json

Revisa el `eas.json` generado vs. el `solucion/eas.json` del repositorio. ¿Qué diferencias encuentras?

4 Lanza tu primera build

Ejecuta `eas build --platform android --profile preview` y sigue el progreso en expo.dev.

5 Bonus: sube a testers

Sube el APK generado a un grupo de testers internos en Play Console.

Resumen del Módulo 6 y el curso

Estos son los conceptos clave que has trabajado en el módulo final. Dominarlos te permite llevar cualquier app React Native desde el código hasta las manos de los usuarios.

Herramienta	Uso	Para qué sirve
eas-cli	<code>npm install -g eas-cli</code>	Instalar la CLI de EAS para compilar y publicar apps desde la terminal.
eas build	<code>eas build --platform android --profile preview</code>	Compilar la app en la nube sin necesitar Android Studio ni Xcode localmente.
eas.json	Perfiles development, preview, production	Configurar los distintos entornos de compilación (debug, APK, AAB).
app.json	package, bundleIdentifier, versionCode	Definir los identificadores únicos de la app antes de publicar en tiendas.
eas submit	<code>eas submit --platform android</code>	Subir el build directamente a Google Play Store o Apple App Store.



¡Has completado el taller de React Native! De `npx create-expo-app` hasta publicar en tiendas. El código de la PokeApp queda como referencia en el repositorio. 🚀

Próximos pasos

El taller cubre los fundamentos esenciales de React Native con Expo. Aquí tienes las tecnologías más relevantes para continuar tu aprendizaje y llevar tus apps al siguiente nivel.



Expo Router

Navegación basada en ficheros al estilo de Next.js – estructura de rutas declarativa y automática.



Reanimated 3

Animaciones más potentes y fluidas que corren en el hilo de UI – sin bloquear el hilo de JavaScript.



Zustand / Jotai

Gestión de estado global ligera y minimalista – sin el boilerplate de Redux.



React Query / SWR

Fetching, caché y sincronización de datos del servidor con reintentos y estado de carga automáticos.



Detox

Tests de integración end-to-end sobre el dispositivo real o simulador – prueba la app como un usuario.